Aula 13: Arquitetura de Computadores – Computadores com conjunto reduzido de instruções

Prof. Hugo Puertas de Araújo
hugo.puertas@ufabc.edu.br
Sala: 509-2 (5º andar / Torre 2)

Arquitetura de Computadores

Objetivos de aprendizagem

- Proporcionar uma visão geral dos resultados de pesquisa sobre as características de execução de instruções que motivaram o desenvolvimento da abordagem RISC.
- Resumir as características-chave das máquinas RISC.
- Entender as implicações de projeto e desempenho ao usar um grande banco de registradores.
- Compreender o uso de otimização de registradores baseada em compilador para melhorar o desempenho.
- Discutir a implicação de uma arquitetura RISC para projeto e desempenho de pipeline.
- Listar e explicar as abordagens-chave para a otimização de pipeline em uma máquina RISC.

Avanços desde os anos 1950

- **Conceito de família:** introduzido pela IBM com seu System/360 em 1964, seguido logo depois por DEC com seu PDP-8. **O conceito de família separa a arquitetura de uma máquina da sua implementação**. Um conjunto de computadores é oferecido, com características de preço/desempenho diferentes e que apresentam a mesma arquitetura ao usuário. As diferenças em preço e desempenho se devem às diferentes implementações da mesma arquitetura.

- **Unidade de controle microprogramada:** sugerida por Wilkes em 1951 e introduzida pela IBM na linha S/360 em 1964. A microprogramação facilita a tarefa de projetar e implementar a unidade de controle e fornece suporte para o conceito de família.

- **Memória cache:** introduzida pela primeira vez comercialmente no S/360 Model 85 da IBM em 1968. A inserção desse elemento na hierarquia de memória melhora o desempenho de modo considerável.

- **Pipeline:** uma forma de introduzir paralelismo na natureza essencialmente sequencial de um programa de instruções de máquina. São exemplos: pipeline de instruções e processamento vetorial.

- **Múltiplos processadores:** essa categoria cobre um número de organizações e objetivos diferentes.

- **Arquitetura de computadores com conjunto reduzido de instruções (RISC)**

Características da execução de instruções

- Características de alguns processadores CISC e RISC:

	Computadores com conjunto complexo de instruções (CISC)			Computadores com conjunto reduzido de instruções (RISC)	
Característica	IBM 370/168	VAX 11/780	Intel 80486	SPARC	MIPS R4000
Ano de desenvolvimento	1973	1978	1989	1987	1991
Número de instruções	208	303	235	69	94
Tamanho da instrução (bytes)	2–6	2–57	1–11	4	4
Modos de endereçamento	4	22	11	1	1
Número de registradores de uso geral	16	16	8	40–520	32
Tamanho de memória de controle (kbits)	420	480	246	—	—
Tamanho de cache (kB)	64	64	8	32	128

Características da execução de instruções

- Características de alguns processadores superescalares:

	Superescalares		
Característica	PowerPC	Ultra SPARC	MIPS R10000
Ano de desenvolvimento	1993	1996	1996
Número de instruções	225		
Tamanho da instrução (bytes)	4	4	4
Modos de endereçamento	2	1	1
Número de registradores de uso geral	32	40–520	32
Tamanho de memória de controle (kbits)	—	—	—
Tamanho de cache (kB)	16–32	32	64

Características principais dos RISC

- Um grande número de registradores de propósito geral e/ou o uso de tecnologia de compiladores para otimizar o uso de registradores.
- Um conjunto de instruções simples e limitado.
- Uma ênfase na otimização do pipeline de instruções.

CMCC
Centro de Matemática, Computação e Cognição

Características da execução de instruções

- Para entender a linha de raciocínio dos defensores do RISC, começamos com uma breve revisão das características da execução de instruções.
- Os aspectos de interesse computacional são:
  i. **Operações efetuadas**: estas determinam as funções a serem efetuadas pelo processador e a sua interação com a memória.
  ii. **Operandos usados**: os tipos de operandos e a frequência do seu uso determinam a organização da memória para armazená-los e os modos de endereçamento para acessá-los.

CMCC Centro de Matemática, Computação e Cognição

Características da execução de instruções

- Frequência dinâmica relativa ponderada das linguagens de programação de alto nível (PATTERSON E SEQUIN, 1982a):

	Ocorrência dinâmica		Instruções de máquina ponderada		Referências à memória ponderada	
	Pascal	**C**	**Pascal**	**C**	**Pascal**	**C**
ATRIBUIÇÃO	45%	38%	13%	13%	14%	15%
LOOP	5%	3%	42%	32%	33%	26%
CHAMADA	15%	12%	31%	33%	44%	45%
IF	29%	43%	11%	21%	7%	13%
GOTO	—	3%	—	—	—	—
OUTRO	6%	1%	3%	1%	2%	1%

Características da execução de instruções

- O estudo de Patterson e Sequin (1982a) analisou também a frequência dinâmica de ocorrência de classes de variáveis.
- Porcentagem dinâmica de operandos:

	Pascal	C	Média
Constante inteira	16%	23%	20%
Variável escalar	58%	53%	55%
Array/estrutura	26%	24%	25%

Características da execução de instruções

- Vimos que as chamadas e os retornos de procedimento são um aspecto dos programas em alto nível.
- Argumentos de procedimentos e variáveis escalares locais:

Porcentagem de chamadas de procedimento executadas com:	Compiladores, interpretadores e editores de texto	Pequenos programas não numéricos
> 3 argumentos	0–7%	0–5%
> 5 argumentos	0–3%	0%
> 8 palavras de argumentos e escalares locais	1–20%	0–6%
> 12 palavras de argumentos e escalares locais	1–6%	0–3%

CMCC Centro de Matemática, Computação e Cognição

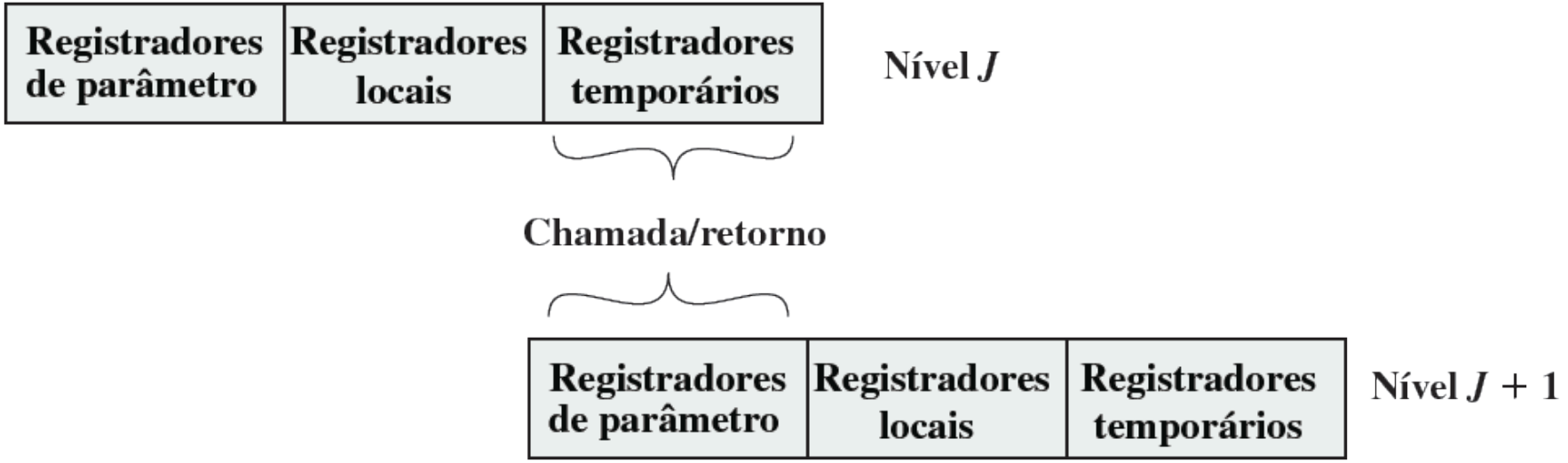
O uso de um grande banco de registradores

- A razão pela qual o armazenamento em registradores é indicado é que esse é o dispositivo de armazenamento mais rápido disponível, mais rápido que a memória principal e que a cache.
- Duas abordagens básicas são possíveis:
  i. A **abordagem em software** depende do compilador para maximizar o uso de registradores.
  ii. A **abordagem de hardware** é simplesmente usar mais registradores para que mais variáveis possam ser guardadas em registradores por mais tempo.

CMCC
Centro de Matemática, Computação e Cognição

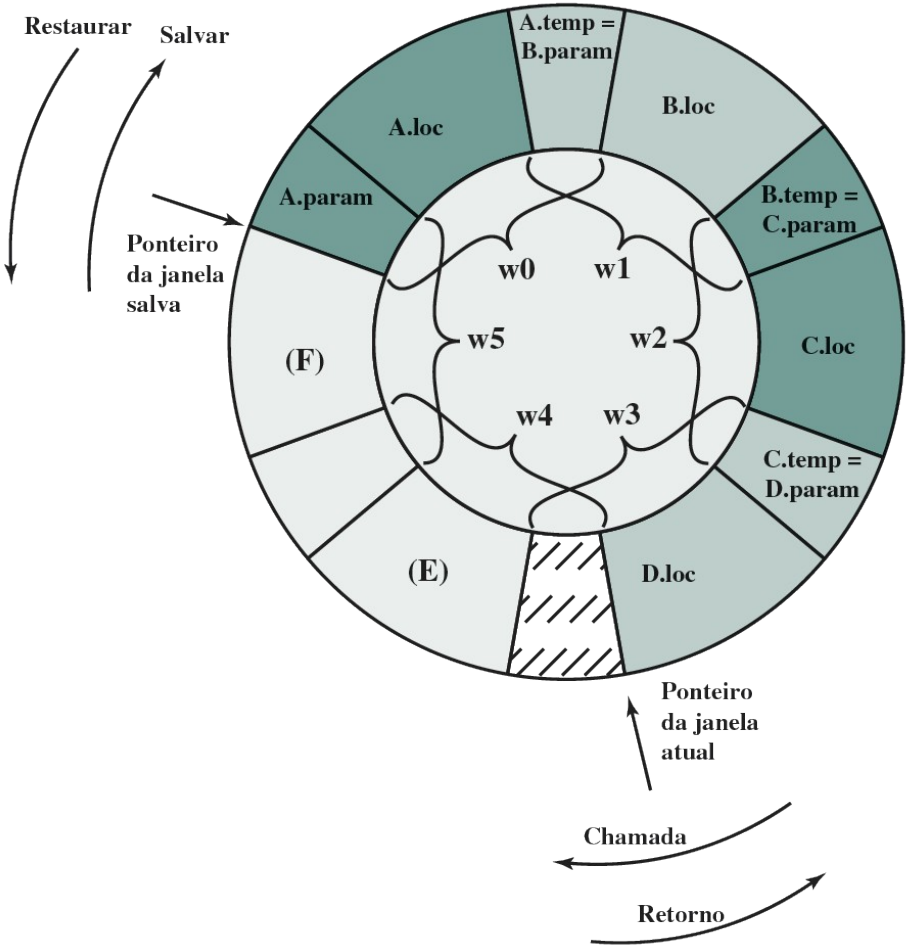
Janelas de registradores

- Sobreposição das janelas de registradores:



Janelas de registradores

- Organização do buffer circular de janelas sobrepostas:



Variáveis globais

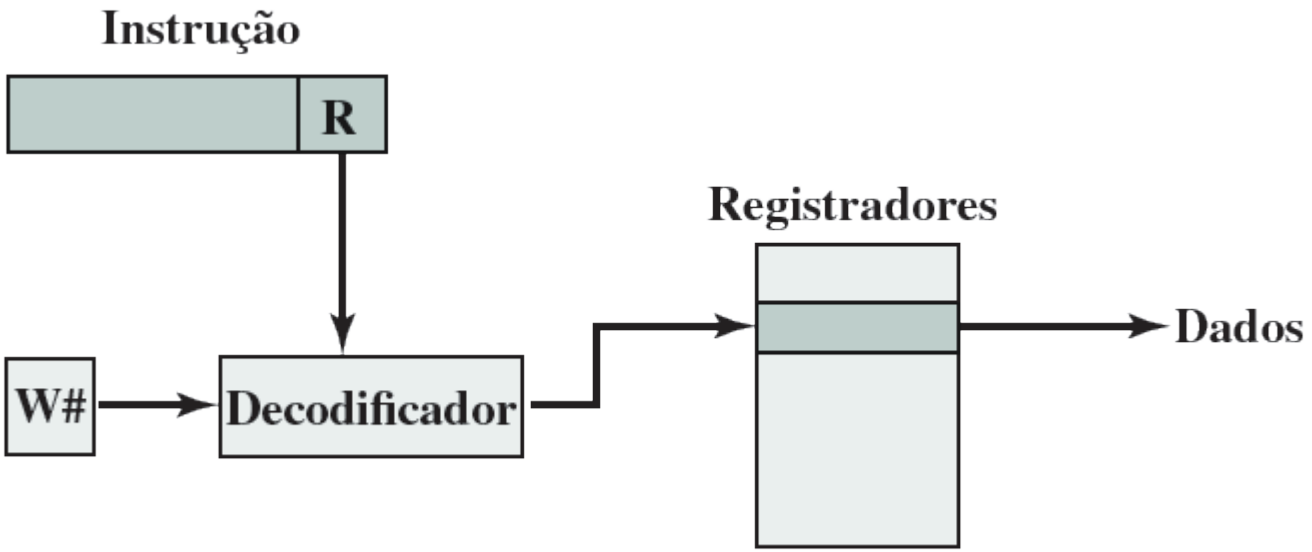
- Às variáveis declaradas como globais, em uma linguagem de alto nível, podem ser atribuídas posições de memória pelo compilador e todas as instruções de máquina que referenciam essas variáveis usarão operandos referenciados em memória.
- Uma alternativa é incorporar um conjunto de registradores globais no processador.
- Esses registradores seriam em número fixo e estariam disponíveis para todos os procedimentos.
- Um esquema uniforme de numeração pode ser usado para simplificar o formato da instrução.

Grande banco de registradores versus cache

Grandes bancos de registradores	Cache
Todas variáveis locais escalares	Variáveis locais recentemente usadas
Variáveis individuais	Blocos de memória
Variáveis globais assinaladas pelo compilador	Variáveis globais recentemente usadas
Salvar/restaurar baseados na profundidade de aninhamento do procedimento	Salvar/restaurar baseado em algoritmos de atualização da cache
Endereçamento de registrador	Endereçamento de memória
Múltiplos operandos endereçados e acessados em um ciclo	Um operando endereçado e acessado por ciclo

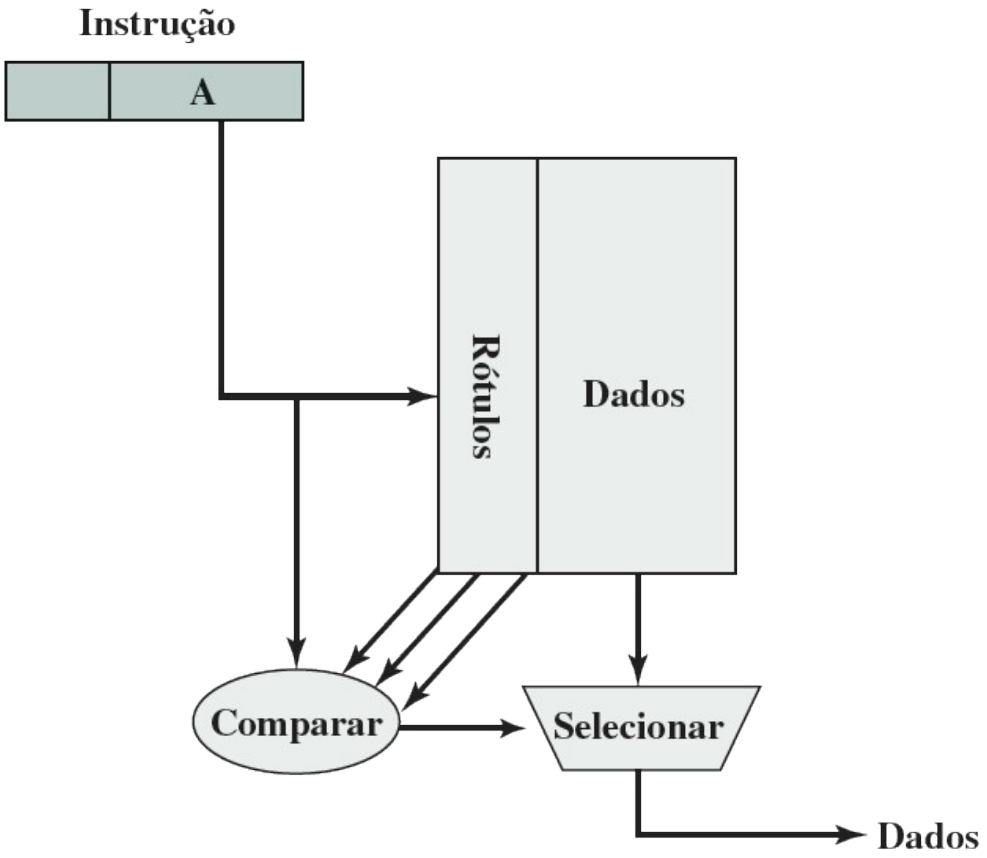
Grande banco de registradores versus cache

- Banco de registradores baseado em janelas:



Grande banco de registradores versus cache

- Cache:



Otimização de registradores baseada em compiladores

- Vamos supor que apenas um número pequeno de registradores esteja disponível na máquina RISC alvo.
- Nesse caso, o uso otimizado de registradores é de responsabilidade do compilador.
- O objetivo do compilador é guardar os operandos nos registradores durante o máximo de operações possível, em vez de usar memória, e minimizar operações de carregar-e-armazenar.
- Em geral, a abordagem usada é a seguinte:

CMCC Centro de Matemática, Computação e Cognição

Otimização de registradores baseada em compiladores

- Cada dado do programa que é candidato a residir em um registrador é atribuído um registrador simbólico ou virtual.
- O compilador, então, mapeia o número ilimitado de registradores simbólicos para um número fixo de registradores reais.
- Registradores simbólicos cujo uso não se sobrepõe podem compartilhar o mesmo registrador real.
- As instruções load/store (carregar/armazenar) são usadas para posicionar os dados nos registradores temporariamente para operações computacionais.

Otimização de registradores baseada em compiladores

- A essência da tarefa de otimização é decidir quais dados devem ser atribuídos aos registradores em qualquer ponto do programa.
- A técnica mais comumente usada em compiladores RISC é conhecida como **coloração de grafos**.
- Primeiro, o programa é analisado para construir um grafo de interferência entre registradores.
- Os nós do grafo são registradores simbólicos.

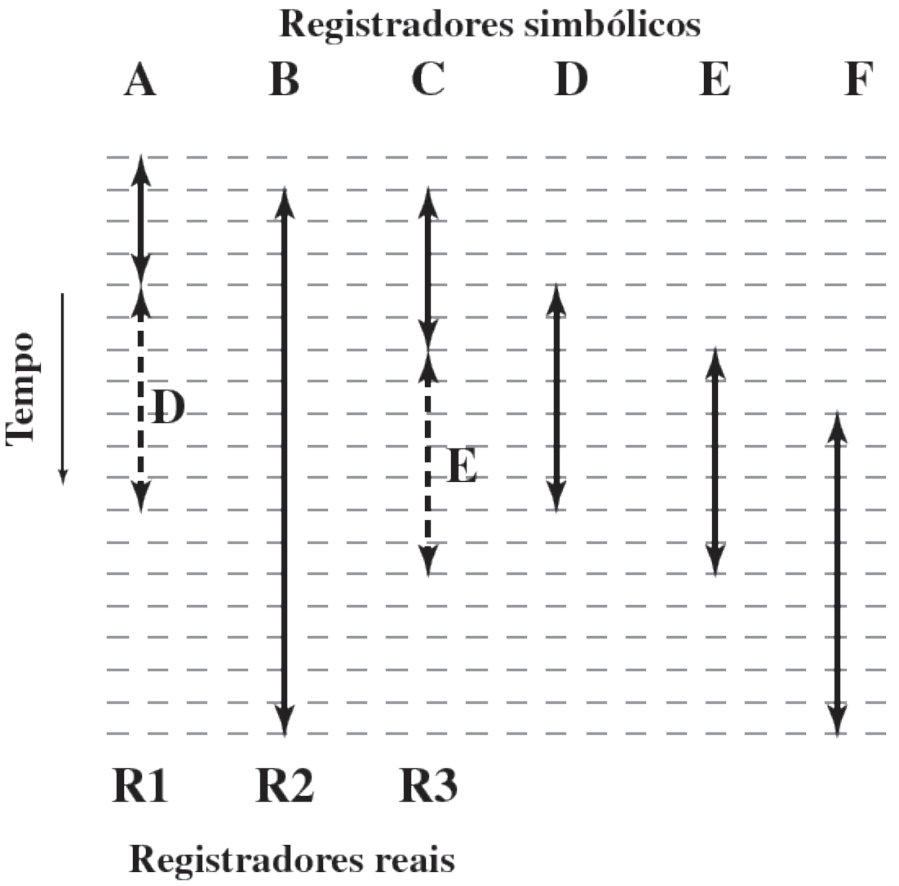
Otimização de registradores baseada em compiladores

- Se dois registradores simbólicos estão “vivos” durante o mesmo fragmento do programa, então eles são unidos por uma linha para demonstrar uma interferência.
- Uma tentativa é feita então para colorir o grafo com n cores, em que n é o número de registradores.
- Nós que compartilham a mesma cor podem ser atribuídos ao mesmo registrador.
- Se esse processo não for completado totalmente, então esses nós que não podem ser coloridos devem ser colocados em memória, e cargas e armazenamentos devem ser usados para criar espaço para os dados quando eles forem necessários.

CMCC
Centro de Matemática, Computação e Cognição

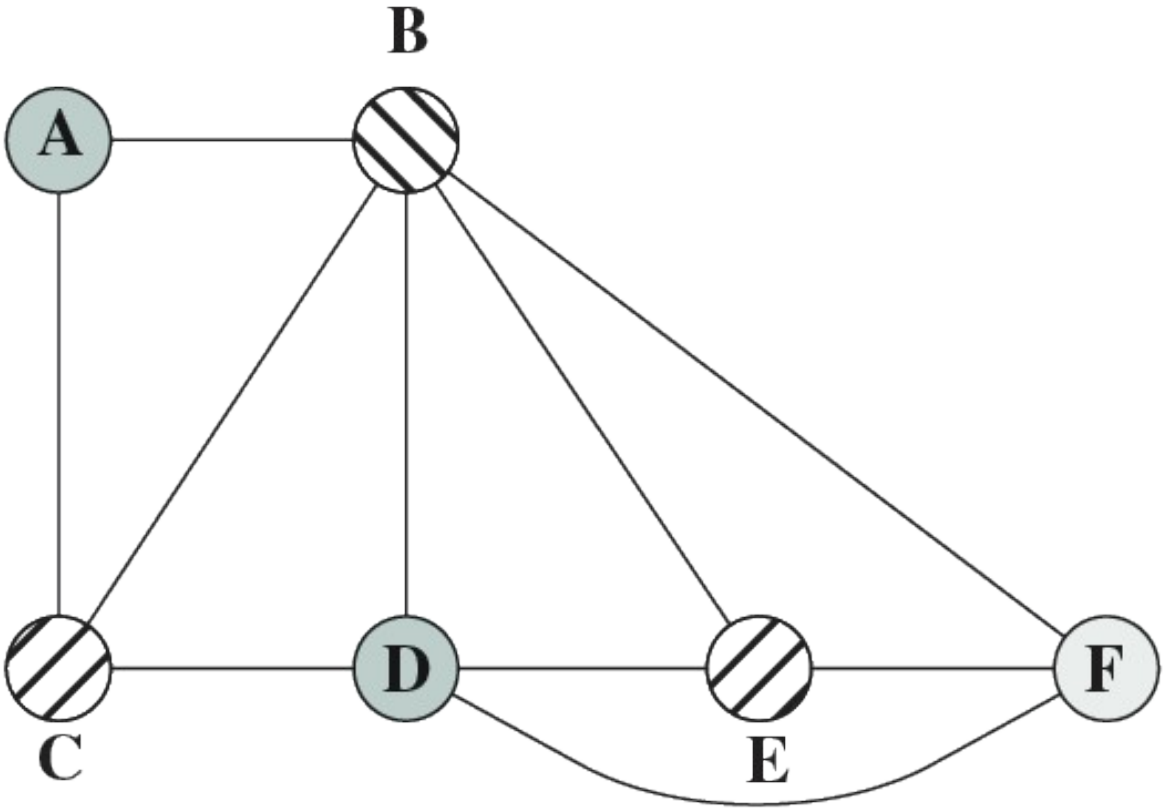
Otimização de registradores baseada em compiladores

- Abordagem de coloração de grafos – Sequência de tempo do uso ativo de registradores:



Otimização de registradores baseada em compiladores

- Abordagem de coloração de grafos – Grafo de interferência de registradores:



Arquitetura com conjunto reduzido de instruções

- Embora várias abordagens para arquitetura com conjunto reduzido de instruções tenham sido implementadas, algumas características são comuns a todas elas:
  - Uma instrução por ciclo.
  - Operações registrador para registrador.
  - Modos de endereçamento simples.
  - Formatos de instruções simples.

Arquitetura com conjunto reduzido de instruções

- Um **ciclo de máquina** é definido como o tempo necessário para obter dois operandos dos registradores, executar uma operação da ALU e armazenar o resultado em um registrador.
- Com instruções simples de ciclo único, há pouca ou nenhuma necessidade de microcódigo.
- A segunda característica é que a maioria das operações deve ser **registrador para registrador**, com apenas operações simples LOAD e STORE acessando a memória.
- Esse recurso de projeto simplifica o conjunto de instruções e, portanto, também a unidade de controle.

Arquitetura com conjunto reduzido de instruções

- A terceira característica é o uso de **modos de endereçamento simples**.
- Esse recurso de projeto simplifica o conjunto de instruções e a unidade de controle.
- A característica comum final é o uso de **formatos de instruções simples**.
- Em geral, apenas um ou alguns poucos formatos são usados.
- O tamanho da instrução é fixo e ajustado dentro do limite da palavra.

Arquitetura com conjunto reduzido de instruções

- A posição de campos, sobretudo o opcode, é fixa.
- Esse recurso tem uma série de benefícios.
- Com campos fixos, a decodificação de opcode e o acesso a registradores de operandos podem ocorrer ao mesmo tempo.
- Formatos simples simplificam a unidade de controle.
- A leitura de instruções é otimizada, porque são obtidas unidades do tamanho da palavra.
- Uma única instrução não ultrapassa os limites da página.

Características CISC versus RISC

- Características de alguns processadores:

Processador	Número de tamanho das instruções	Tamanho máximo da instrução em bytes	Número de modos de endereçamento	Endereçamento indireto	Load/store combinada com aritmética	Número máximo de operandos de memória	Endereçamento desalinhado permitido	Número máximo de usos da MMU	Número de bits para especificadores de registradores inteiros	Número de bits para especificadores de registradores de ponto flutuante
AMD29000	1	4	1	não	não	1	não	1	8	3[a]
MIPS R2000	1	4	1	não	não	1	não	1	5	4
SPARC	1	4	2	não	não	1	não	1	5	4
MC88000	1	4	3	não	não	1	não	1	5	4
HP PA	1	4	10[a]	não	não	1	não	1	5	4
IBM RT/PC	2[a]	4	1	não	não	1	não	1	4[a]	3[a]
IBM RS/6000	1	4	4	não	não	1	sim	1	5	5
Intel i860	1	4	4	não	não	1	não	1	5	4
IBM 3090	4	8	2[b]	não[b]	sim	2	sim	4	4	2
Intel 80486	12	12	15	não[b]	sim	2	sim	4	3	3
NSC 32016	21	21	23	sim	sim	2	sim	4	3	3
MC68040	11	22	44	sim	sim	2	sim	8	4	3
VAX	56	56	22	sim	sim	6	sim	24	4	0
Clipper	4[a]	8[a]	9[a]	não	não	1	0	2	4[a]	3[a]
Intel 80960	2[a]	8[a]	9[a]	não	não	1	sim[a]	—	5	3[a]

Notas:

[a] RISC que não está conforme a essa característica.

[b] CISC que não está conforme a essa característica.

CMCC
Centro de Matemática, Computação e Cognição

Pipeline no RISC

- Pipeline com instruções regulares
  - A maioria das instruções é do tipo registrador para registrador e o ciclo de instruções possui os dois estágios a seguir:
    i. I: busca da instrução.
    ii. E: execução.
  - Para operações de carregar e armazenar:
    i. I: busca da instrução.
    ii. E: execução.
    iii. D: memória.

CMCC Centro de Matemática, Computação e Cognição

Pipeline no RISC

- Pipeline com instruções regulares
  - Os efeitos de pipeline:

13 ciclos de máquina

Load rA ← M	I	E	D										
Load rB ← M				I	E	D							
Add rC ← rA + rB							I	E					
Store M ← rC									I	E	D		
Branch X												I	E

Execução sequencial

10 ciclos de máquina

Load rA ← M	I	E	D							
Load rB ← M		I		E	D					
Add rC ← rA + rB				I		E				
Store M ← rC						I	E	D		
Branch X							I		E	
NOOP									I	E

Temporização do pipeline em dois estágios

Pipeline no RISC

- Pipeline com instruções regulares
  - Os efeitos de pipeline:

Load	rA ← M	I	E	D				
Load	rB ← M		I	E	D			
NOOP				I	E			
Add	rC ← rA + rB				I	E		
Store	M ← rC					I	E	D
Branch X							I	E
NOOP								I

Temporização do pipeline em três estágios

Load	rA ← M	I	E_1	E_2	D						
Load	rB ← M		I	E_1	E_2	D					
NOOP				I	E_1	E_2					
NOOP					I	E_1	E_2				
Add	rC ← rA + rB					I	E_1	E_2			
Store	M ← rC						I	E_1	E_2	D	
Branch X								I	E_1	E_2	
NOOP									I	E_1	E_2
NOOP										I	E_1	E_2

Temporização do pipeline em quatro estágios

Pipeline no RISC

- Otimização de pipeline
  - **DESVIO ATRASADO**: Para compensar essas dependências, técnicas de reorganização de código foram desenvolvidas.
  - O desvio atrasado (delayed branch), uma maneira de aumentar a eficiência do pipeline, faz uso de um desvio que não tem efeito até depois da execução da instrução seguinte (por isso o termo atrasado).
  - A posição da instrução imediatamente depois do desvio é referida como slot de atraso (delay slot).

CMCC
Centro de Matemática, Computação e Cognição

Pipeline no RISC

- Otimização de pipeline
  - Desvio normal e atrasado:

Endereço	Desvio normal		Desvio atrasado		Desvio atrasado otimizado	
100	LOAD	X, rA	LOAD	X, rA	LOAD	X, rA
101	ADD	1, rA	ADD	1, rA	JUMP	105
102	JUMP	105	JUMP	106	ADD	1, rA
103	ADD	rA, rB	NOOP		ADD	rA, rB
104	SUB	rC, rB	ADD	rA, rB	SUB	rC, rB
105	STORE	rA, Z	SUB	rC, rB	STORE	rA, Z
106			STORE	rA, Z		

Pipeline no RISC

- Otimização de pipeline
  - Uso do desvio atrasado – Pipeline tradicional:

Tempo →

	1	2	3	4	5	6	7	8
100 LOAD X, rA	I	E	D					
101 ADD 1, rA		I		E				
102 JUMP 105				I	E			
103 ADD rA, rB					I	E		
105 STORE rA, Z						I	E	D

Pipeline no RISC

- Otimização de pipeline
  - Uso do desvio atrasado – Pipeline RISC com adição de NOOP:

	1	2	3	4	5	6	7	8
100 LOAD X, rA	I	E	D					
101 ADD 1, rA		I		E				
102 JUMP 106				I	E			
103 NOOP					I	E		
106 STORE rA, Z						I	E	D

Pipeline no RISC

- Otimização de pipeline
  - Uso do desvio atrasado – Instruções invertidas:

	1	2	3	4	5	6
100 LOAD X, Ar	I	E	D			
101 JUMP 105		I	E			
102 ADD 1, rA			I	E		
105 STORE rA, Z				I	E	D

Pipeline no RISC

- Otimização de pipeline
  - **LOAD ATRASADO**: Uma tática semelhante, chamada load atrasado (delayed load), pode ser usada em instruções LOAD.
  - O registrador é bloqueado pelo processador.
  - O processador então continua a execução do fluxo de instruções até que alcance uma instrução que precisa desse registrador.
  - Se o compilador puder rearranjar as instruções para que um trabalho útil possa ser feito enquanto ocorre leitura dentro do pipeline, a eficiência será aumentada.

CMCC
Centro de Matemática, Computação e Cognição

Pipeline no RISC

- Otimização de pipeline
  - **DESDOBRAMENTO DE LOOP**: Desdobrar replica o corpo de um loop em um número de vezes chamado de fator de desdobramento (u) e faz a iteração pelo passo u em vez de pelo passo 1.
  - Desdobrar pode melhorar o desempenho:
    i. Reduzindo a sobrecarga do loop.
    ii. Aumentando o paralelismo de instruções.
    iii. Melhorando a localidade de registradores, cache de dados ou TLB.

CMCC Centro de Matemática, Computação e Cognição

MIPS R4000

- Um dos primeiros chips RISC disponível comercialmente foi desenvolvido pela MIPS Technology Inc.
- Nesta seção, analisamos o MIPS R4000.
- Ele possui substancialmente a mesma arquitetura e o mesmo conjunto de instruções de projetos MIPS anteriores: R2000 e R3000.
- A diferença mais significativa é que o R4000 usa 64 em vez de 32 bits para os caminhos de dados internos e externos, endereços, registradores e ALU.

CMCC
Centro de Matemática, Computação e Cognição

MIPS R4000

- Ele permite um espaço de endereçamento maior.
- O chip do processador R4000 é particionado em duas seções, uma contendo CPU e outra contendo um coprocessador para gerenciamento de memória.
- O processador possui uma arquitetura muito simples.
- O processador suporta 32 registradores de 64 bits.
- Ele também possui uma cache de alta velocidade de 128 kB, sendo metade para instruções e metade para dados.

MIPS R4000

- Todas as instruções de série MIPS R são codificadas em um formato de palavra de 32 bits.
- Todas as operações de dados são de registrador para registrador.
- As únicas referências de memória são as operações puramente de load/store.
- O R4000 não faz uso de códigos condicionais.
- O MIPS usa um tamanho único da instrução de 32 bits.

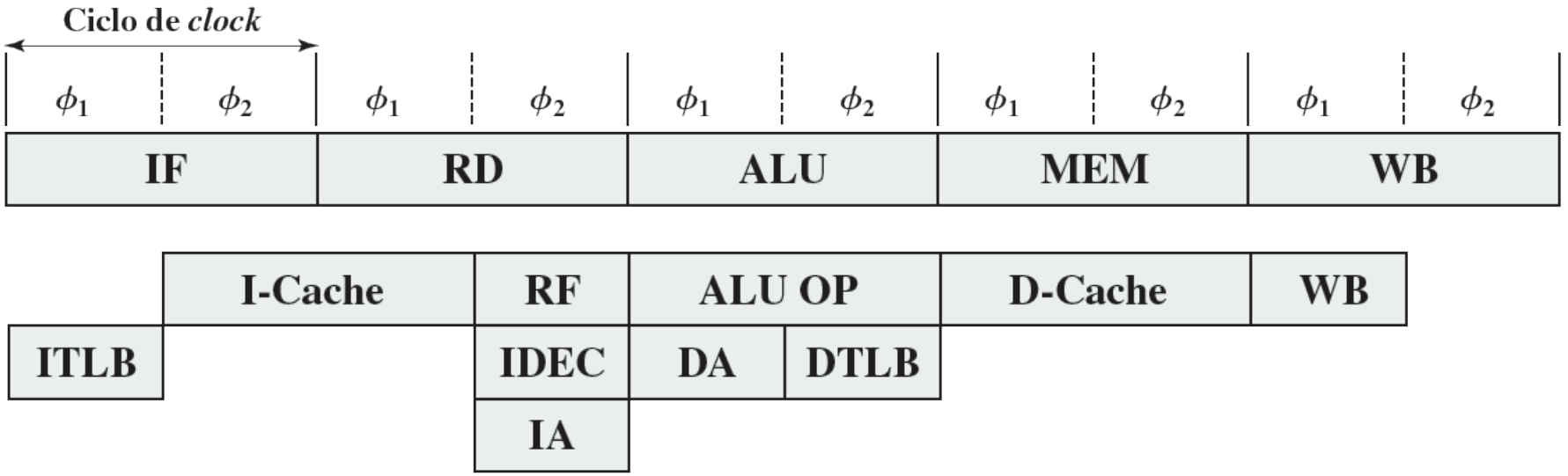
MIPS R4000

- Cada um dos 32 registradores de uso geral pode ser usado como registrador base.
- Um registrador, r0, sempre contém 0.
- O compilador faz uso de múltiplas instruções de máquina para sintetizar modos de endereçamento típicos em máquinas convencionais.
- O MIPS pode alcançar uma eficiência grande em seu pipeline.
- A figura a seguir mostra o pipeline de instruções do R3000.

CMCC
Centro de Matemática, Computação e Cognição

MIPS R4000

- Pipeline do R3000 detalhado:



MIPS R4000

- Melhoria do pipeline do R3000:

Ciclo	Ciclo	Ciclo	Ciclo	Ciclo	Ciclo
ITLB	I-Cache	RF	ALU	DTLB	D-Cache	WB

Pipeline do R3000 modificado com latências reduzidas

Ciclo	Ciclo	Ciclo	Ciclo	Ciclo
ITLB	RF	ALU	D-Cache	TC	WB

Pipeline do R3000 otimizado com acesso paralelo a TLB e cache

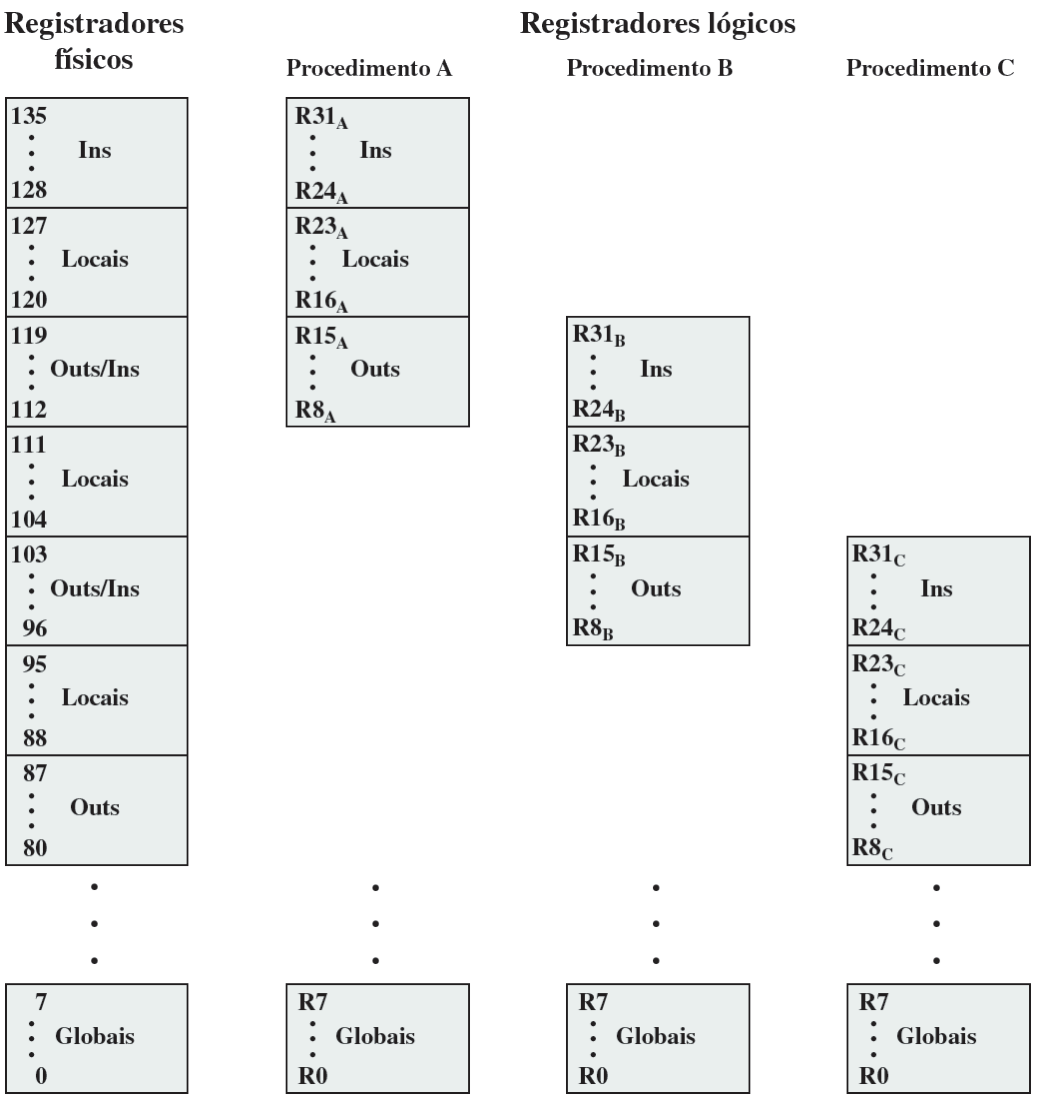
IF = Busca da instrução
RD = Leitura
MEM = Acesso à memória
WB = Atualização
I-Cache = Acesso à cache de instruções
RF = Busca do operando do registrador
D-Cache = Acesso à cache de dados
ITLB = Tradução do endereço da instrução
IDEC = Decodificação da instrução
IA = Cálculo do endereço da instrução
DA = Cálculo do endereço virtual de dados
DTLB = Tradução do endereço de dados
TC = Verificação do rótulo de cache de dados

MIPS R4000

- O R4000 possui oito estágios de pipeline, o que significa que no máximo oito instruções podem estar no pipeline ao mesmo tempo:
  i. Busca da instrução
  ii. Busca da instrução
  iii. Banco de registradores
  iv. Execução de instrução
  v. Primeira cache de dados
  vi. Segunda cache de dados
  vii. Verificação de rótulos
  viii. Atualização

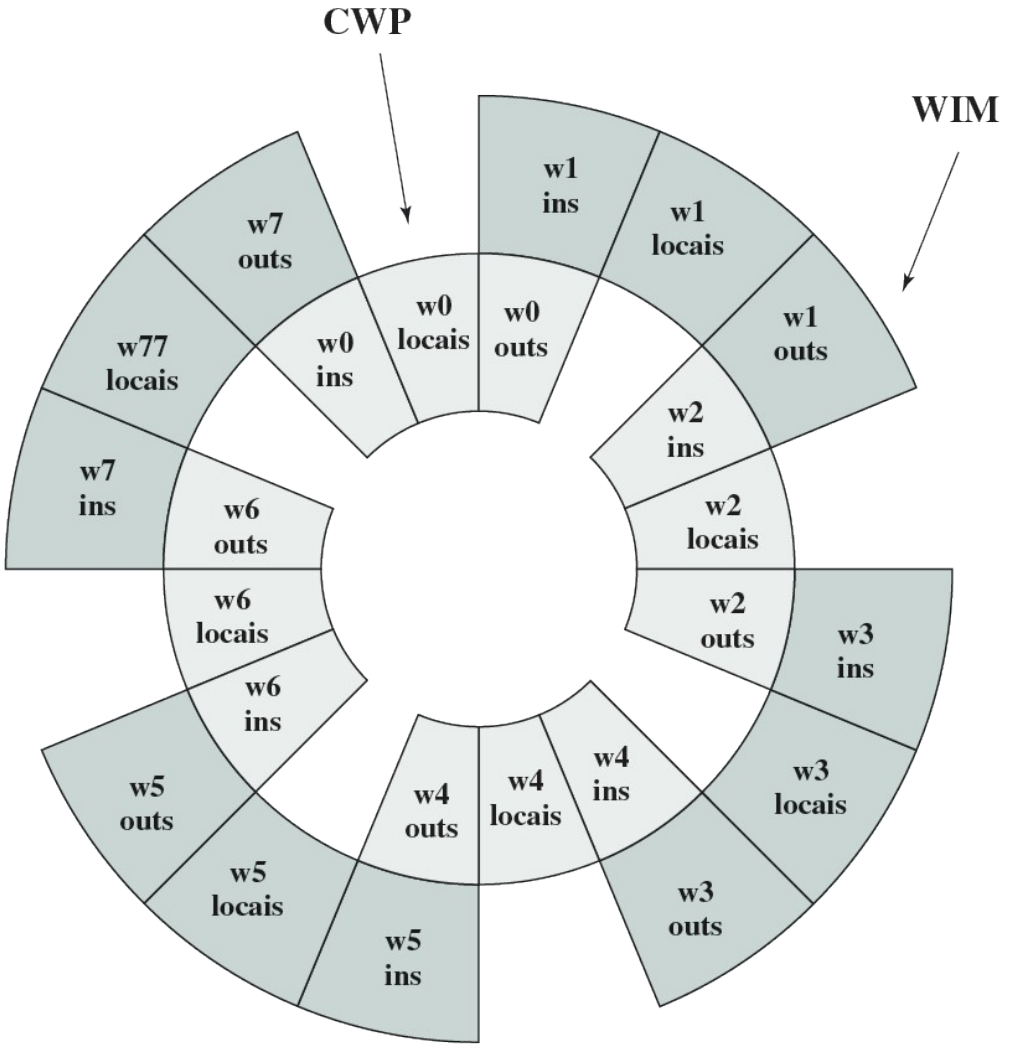
SPARC

- Assim como acontece com o RISC de Berkeley, a SPARC também faz uso de janelas de registradores.



SPARC

- Janelas com oito registradores formando uma pilha circular no SPARC:



SPARC

- Todas as instruções, exceto os deslocamentos, podem opcionalmente definir quatro códigos condicionais (ZERO, NEGATIVO, OVERFLOW, CARRY).
- Inteiros com sinal são representados na forma de complemento de dois com 32 bits.
- Apenas instruções simples de carregar e armazenar referenciam a memória.
- Existem instruções diferentes de carregar e armazenar para palavra (32 bits), palavras duplas, meias palavras e bytes.

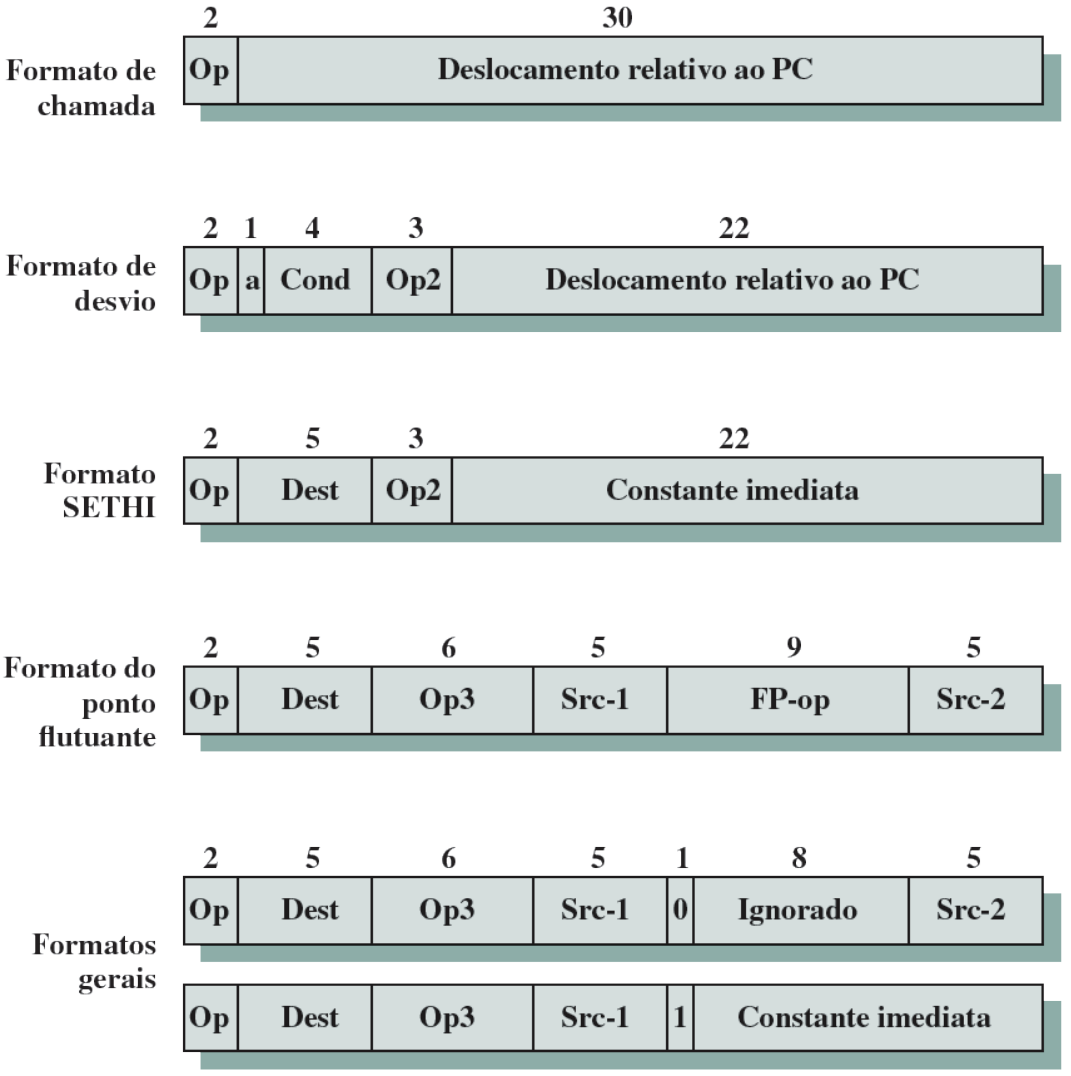
CMCC
Centro de Matemática, Computação e Cognição

SPARC

- Para os dois últimos casos, existem instruções para carregar essas grandezas como números com ou sem sinal.
- Números com sinal são estendidos para preencher o registrador de destino de 32 bits.
- Números sem sinal são completados com zeros.
- O único modo de endereçamento disponível, além do modo de registrador, é o modo por deslocamento.
- Para executar um load ou store, um estágio extra é adicionado ao ciclo de instrução.

SPARC

- O SPARC usa um conjunto simples de formatos da instrução de 32 bits:



Controvérsia de RISC versus CISC

- Em anos mais recentes, a controvérsia RISC versus CISC sumiu quase que totalmente.
- Isso aconteceu porque houve uma convergência gradual das tecnologias.
- Com o aumento da densidade dos chips e da velocidade do hardware, os sistemas RISC se tornaram mais complexos.
- Ao mesmo tempo, os modelos CISC focaram em questões tradicionalmente associadas com RISC, como um aumento no número de registradores de uso geral e mais ênfase no projeto do pipeline de instruções.